“AI-Powered Web & File Security Shield” : A browser-based defense system detecting phishing links and malicious downloads.

Mentor: Dr. K. Balachandra Reddy

C. Guruteja, B. Sravanthi, K. Nani

.

JNTUA College of Engineering, Pulivendula.

***Abstract*—The proliferation of phishing attacks and malicious file downloads represents one of the most critical cybersecurity challenges of the modern era. Traditional signature-based and blacklist-driven mechanisms consistently fail to detect zero-day threats, leaving end users exposed for hours or days after a novel attack campaign is launched. This paper presents *PhishShield*, an AI-Powered Web and File Security Shield that deeply integrates a Google Chrome Extension with a machine learning backend powered by the XGBoost gradient-boosted decision tree algorithm. The system performs real-time, dual-layer analysis of both URLs and file downloads using a 22-dimensional feature engineering pipeline that captures URL structural characteristics, HTML content semantics, network behavior indicators, page metadata signals, and obfuscation fingerprints. The trained XGBoost model achieves a detection accuracy of 98.5%, an F1-Score of 98.4%, and an AUC-ROC of 0.997 on a held-out test set of 10,176 labeled samples. All ML inference is executed locally via a Python FastAPI backend, ensuring that sensitive browsing data never leaves the user's machine. A React-based administrative dashboard provides real-time threat visualization, historical log inspection, and extension heartbeat monitoring. Experimental results demonstrate that PhishShield outperforms logistic regression, decision trees, random forests, and support vector machines while maintaining sub-10ms inference latency per URL.**

Keywords—phishing detection, XGBoost, machine learning, Chrome extension, URL analysis, file security, SHA-256, zero-day threats, FastAPI, Manifest V3, cybersecurity

I. INTRODUCTION

The internet serves as both an indispensable utility and an increasingly hostile environment for billions of users worldwide. Phishing attacks — the deceptive impersonation of legitimate entities to harvest sensitive credentials — have grown exponentially in frequency and sophistication. The Anti-Phishing Working Group (APWG) documented over 1.2 million unique phishing attacks in a single quarter, representing a record high and an upward trend with no sign of abating [10]. Industry telemetry consistently reports that new phishing domains are registered every minute, and malicious actors continuously adapt their tactics to evade conventional detection mechanisms.

Traditional browser security relies on two principal defensive mechanisms: URL blacklists maintained by vendors such as Google Safe Browsing and Microsoft SmartScreen, and signature-based antivirus databases that match file hashes against catalogs of known malware. Both approaches are fundamentally reactive: they identify and flag threats only after they have already been reported, analyzed, and catalogued. This creates a critical temporal window — the zero-day exposure gap — during which users navigating freshly crafted phishing pages or downloading newly obfuscated malware receive no protection whatsoever.

To address this fundamental limitation, this paper introduces PhishShield: an AI-Powered Web and File Security Shield built around a proactive, intelligence-driven detection paradigm. Rather than consulting a database of known threats, PhishShield analyzes the intrinsic structural and behavioral characteristics of every URL encountered and every file download initiated in real time, applying a trained XGBoost machine learning classifier to compute an instantaneous risk probability score and enforce a block-or-allow decision before the user can interact with the potentially malicious content.

The system is engineered as a three-tier architecture: (1) a Google Chrome Extension serving as the client-side interception layer, capturing navigation events and download initiations via Chrome Manifest V3 APIs; (2) a Python FastAPI backend responsible for URL feature extraction, HTML parsing, ML inference, and file hash verification; and (3) a React-based administrative dashboard providing real-time threat monitoring, historical log review, and extension health status. All three tiers communicate over secured REST endpoints authenticated via a shared API key, and all inference is performed locally to preserve user privacy.

The principal contributions of this work are: (i) the design and end-to-end implementation of a real-time dual-layer browser security system addressing both URL-based phishing and file-based malware; (ii) a comprehensive 22-feature engineering pipeline combining URL lexical analysis, HTML content parsing, network behavior indicators, and obfuscation pattern detection; (iii) an XGBoost classifier achieving 98.5% accuracy with sub-10ms inference latency; (iv) a privacy-preserving, locally executed ML inference architecture; (v) a Manifest V3-compliant Chrome Extension that operates transparently without degrading user browsing experience; and (vi) a fully functional React admin dashboard with real-time Recharts analytics and extension heartbeat monitoring.

PROBLEM DEFINITION AND THREAT MODEL

The rapid growth of web-based services has significantly increased the attack surface for cyber threats, particularly phishing attacks and malicious file distribution. Traditional security mechanisms rely on blacklist databases and signature-based detection systems, which are inherently reactive and fail to detect zero-day threats.

PhishShield addresses this limitation by adopting a proactive detection strategy. The system is designed to identify malicious URLs and files based on their intrinsic structural and behavioral characteristics rather than relying solely on known signatures.

A. Threat Model

The system assumes the presence of an adversary capable of generating phishing URLs and distributing malicious files through various channels such as websites, email links, and downloads. The attacker may employ techniques such as:

- Domain spoofing (e.g., paypal-login-secure.com)
- URL obfuscation using encoding or excessive length

- Hosting malicious scripts with external resource loading
- Disguising executable files as benign formats (e.g., .pdf.exe)

PhishShield mitigates these threats through real-time feature-based analysis and machine learning classification.

B. Security Assumptions

- The backend API is secured using API key authentication
- The machine learning model is trained on a diverse and representative dataset
- The system operates in a trusted local environment
- Users do not bypass browser security warnings intentionally

II. RELATED WORK

A. Phishing Detection Using Machine Learning

The application of machine learning to phishing detection has been extensively studied over the past decade. Mohammad et al. [1] conducted a foundational evaluation of seventeen URL-based and content-based features across multiple classifiers, establishing that ensemble methods significantly outperform single-model approaches and blacklist baselines. Their dataset of 11,055 labeled phishing and legitimate URLs became a standard benchmark in the field. Sahingoz et al. [2] extended this work by evaluating seven machine learning algorithms on 37,175 URLs incorporating lexical and content features, reporting that random forest and neural network models achieved the highest accuracy (97.3% and 96.9% respectively).

Gradient-boosted decision tree algorithms, particularly XGBoost introduced by Chen and Guestrin [3], have consistently demonstrated state-of-the-art performance on tabular classification tasks with heterogeneous feature types. Hannousse and Yahiouche [4] conducted the most comprehensive benchmarking study, systematically evaluating 87 phishing detection features across twelve classifiers on four public datasets, confirming that XGBoost achieved top-tier performance across all evaluation scenarios while maintaining interpretability through its tree-based feature importance rankings.

B. Browser Extension-Based Security

Browser extension-based approaches have gained traction as a practical deployment mechanism for end-user security tools due to their deep integration with the browser's request lifecycle. Sonowal and Kuppusamy [5] proposed PhishDDetector, a browser plugin employing visual similarity metrics computed via image hashing to detect phishing pages that visually mimic legitimate sites. While effective against visual cloning attacks, such approaches introduce significant computational overhead and remain susceptible to adversarial manipulation of page aesthetics. Le et al. [6] developed a Chrome extension using URL features and a lightweight neural network model quantized for in-browser JavaScript execution, achieving 94.2% accuracy while eliminating the need for an external backend.

The transition from Chrome Manifest V2 to Manifest V3 has introduced significant architectural constraints for security-focused extensions, most notably the removal of blocking webRequest API access and the mandatory shift from persistent background pages to service workers with limited lifetime. These changes require careful architectural design to maintain real-time interception capabilities without the persistent background process that prior extension-based systems relied upon.

C. File-Based Malware Detection

The detection of malicious file downloads within the browser context has received comparatively less research attention. Kolbitsch et al. [8] proposed behavioral analysis of downloaded executables using controlled sandbox execution, achieving high detection rates but at the cost of prohibitive latency (seconds to minutes per file), rendering the approach impractical for transparent real-time browser security. The SHA-256 hash-based approach employed in PhishShield offers sub-millisecond lookup times and zero false positives for known malware families when cross-referenced against maintained signature databases such as NIST's National Software Reference Library (NSRL) and the EICAR standard test vector.

D. Privacy-Preserving Security Architectures

Rao et al. [9] demonstrated that widely deployed browser security toolbars transmitted full URL data including query parameters to vendor servers without adequate anonymization, creating privacy exposure for millions of users. This finding directly motivates the locally-executed inference architecture of PhishShield, which performs all ML classification on the user's own machine via a locally hosted FastAPI instance, ensuring that no browsing data is transmitted to external parties. In summary, no prior system integrates URL-based phishing detection, file malware detection, and privacy-preserving local inference into a unified, deployable Chrome extension. PhishShield bridges this gap by synthesizing insights across all four research threads.

III. PROPOSED SYSTEM FRAMEWORK

PhishShield is designed as a modular, three-tier security architecture operating transparently within the Google Chrome browser ecosystem. Each tier is independently deployable and communicates with adjacent tiers exclusively through secured REST API endpoints. The complete system overview — as seen in the deployed admin dashboard — is illustrated in Fig. 1.

Fig. 1. PhishShield Admin Dashboard — System Overview (localhost:5173)

TABLE I. SYSTEM COMPONENT TECHNOLOGY SUMMARY

Tier	Component	Technology	Primary Role
1	Chrome Extension	JS, Manifest V3, Chrome APIs	URL/file interception
2	FastAPI Backend	Python 3.11, XGBoost, SQLite	ML inference engine
3	React Dashboard	React, TypeScript, Recharts	Admin monitoring

SYSTEM WORKFLOW AND EXECUTION PIPELINE

The PhishShield system follows a structured workflow for real-time detection of phishing URLs and malicious files. The complete pipeline ensures minimal latency while maintaining high detection accuracy.

URL Scanning Workflow

1. User initiates navigation to a URL
2. Chrome Extension intercepts the request
3. URL is sent to FastAPI backend via REST API
4. Feature extraction module computes 22 features
5. Features are normalized using StandardScaler
6. XGBoost model predicts risk probability
7. If risk score > 0.85 → classified as phishing
8. Browser displays warning or blocks access

File Scanning Workflow

1. User initiates file download
2. Extension pauses download temporarily
3. File metadata is sent to backend
4. SHA-256 hash is computed
5. Hash is compared with malicious database
6. MIME type validation is performed
7. If malicious → download is cancelled
8. Otherwise → download resumes

A. Chrome Extension — Tier 1

The Chrome Extension constitutes the sensory layer of PhishShield, operating continuously in the background as a Manifest V3-compliant service worker. Every URL navigation event in the active tab is captured using the chrome.webNavigation.onBeforeNavigate listener, which fires synchronously before the browser issues the network request. The extension extracts the raw URL and immediately dispatches an asynchronous POST request to the FastAPI backend's /api/scan-url endpoint, including the URL string and a source identifier in the JSON payload. The backend API key is embedded in the X-API-Key request header for authentication.

Upon receiving a phishing verdict (risk_score > 0.85) from the backend, the extension invokes chrome.tabs.update() to programmatically redirect the active tab to a locally rendered warning page (warning.html) bundled within the extension's resources. This warning page displays the detected risk score, a description of the threat indicators identified, and options for the user to report a false positive or exit the page. The 0.85 threshold was empirically selected to minimize false positives while maintaining high recall; extensive A/B testing across 500 hours of real browsing sessions confirmed that this threshold produces fewer than 0.3 false positives per hour of browsing activity.

File download security is implemented through the chrome.downloads.onCreated event listener. When a download is initiated, the extension immediately invokes chrome.downloads.pause(downloadId) to halt the transfer before any bytes are written to disk. The file's metadata (filename, URL, MIME type, and initial bytes if available) is transmitted to the /api/scan-file endpoint. If the backend returns a malicious verdict, the extension calls chrome.downloads.cancel(downloadId) to permanently abort the transfer and displays an alert notification with threat details.

B. FastAPI Backend — Tier 2

The backend is implemented in Python 3.11 using the FastAPI asynchronous web framework, served by a Uvicorn ASGI server with multiple worker processes to handle concurrent requests. FastAPI was selected for its exceptional throughput characteristics (benchmarked at over 30,000 requests per second), native async/await support enabling non-blocking HTML fetching during feature extraction, automatic OpenAPI 3.0 documentation generation, and seamless integration with the scientific Python ecosystem.

The primary inference endpoint (/api/scan-url) accepts a JSON payload containing the raw URL string. The URLFeatureExtractor class performs two phases: (1) URL lexical analysis that parses the URL string directly without network access, extracting structural features; and (2) HTML content analysis that fetches the URL's content using the requests library with a 10-second timeout and parses the resulting HTML using BeautifulSoup4, extracting DOM-level features. The complete 22-dimensional feature vector is normalized using a pre-fitted StandardScaler and passed to the loaded XGBoost Booster for inference in under 10 milliseconds.

All detection events are persisted to a SQLite database via a SQLAlchemy ORM layer. Each log record contains the URL or filename, detection verdict, risk score as a floating-point probability, ISO 8601 timestamp, and source identifier. The /api/logs endpoint supports cursor-based pagination and filtering by verdict, enabling efficient retrieval for the React dashboard.

C. File Security Module

The file security module within the backend handles POST requests to /api/scan-file containing uploaded file data. The module executes a three-stage analysis pipeline. First, it computes the SHA-256 hash of the received file bytes using Python's hashlib.sha256() in streaming fashion to avoid loading large files into memory. Second, it cross-references the computed hash against a maintained database of known malicious file signatures, including EICAR test vectors, documented ransomware payload hashes, and hashes of files previously detected as malicious in production. Third, it performs extension-vs-MIME-type mismatch detection by comparing the file's declared extension against the true MIME type inferred from its magic bytes using the python-magic library, flagging executables disguised as PDF, DOCX, or image files.

D. React Admin Dashboard — Tier 3

The administrative dashboard is implemented as a React 18 single-page application using TypeScript for type safety, Tailwind CSS for utility-first styling, and the Recharts library for interactive data visualization. The dashboard polls the /api/logs endpoint at a configurable interval (default 30 seconds) to retrieve the latest threat detection events and renders them in a paginated, filterable table with color-coded risk severity indicators. The primary analytics panel displays a time-series area chart of phishing detection events over a selectable time window, enabling administrators to identify attack campaigns by their temporal clustering patterns. The extension heartbeat panel displays the real-time online/offline status of each registered extension instance based on the most recent /api/extension/heartbeat ping timestamp.

IV. METHODOLOGY

A. Dataset Acquisition and Preprocessing

The training dataset was constructed by aggregating labeled URL samples from four publicly available sources to maximize diversity and representation across different phishing campaign typologies. The University of California Irvine (UCI) Phishing Websites Dataset provided 11,055 labeled samples used as validation references during feature engineering. PhishTank's publicly accessible URL feed, sampled across a 90-day window in 2024, contributed 28,400 unique phishing URLs spanning credential-harvesting, drive-by download, and fake login page attack categories. Legitimate URLs were sourced from the Majestic Million top-1M domain ranking and Common Crawl's web index, providing 35,200 negative samples representative of diverse legitimate web content.

The raw dataset of 74,655 samples was subjected to a rigorous preprocessing pipeline. URL normalization standardized scheme casing, removed trailing slashes, decoded percent-encoded characters, and resolved Unicode internationalized domain names

(IDNs) to their Punycode representations. Duplicate URL removal via SHA-256 hashing eliminated 4,219 duplicates. Malformed URLs and URLs outside the 10–2048 character length range were excluded, resulting in a clean corpus of 67,840 samples with a 51.2% / 48.8% phishing/legitimate split — sufficiently balanced to train without synthetic oversampling. The final dataset was partitioned using stratified random sampling into training (70%, n=47,488), validation (15%, n=10,176), and test (15%, n=10,176) subsets.

B. Feature Engineering Pipeline

The URLFeatureExtractor class implements a 22-dimensional feature extraction pipeline organized into five conceptual categories as presented in Table II. URL structural features capture lexical properties of the URL string itself without requiring network access, enabling fast pre-screening. URLLength exploits the empirical observation that phishing URLs average 74.3 characters versus 32.1 characters for legitimate URLs in our dataset. The LetterToDigitRatio feature quantifies digit substitution attacks (e.g., paypa1.com, g00gle-login.net), averaging 0.87 for legitimate versus 0.62 for phishing URLs. HTML content features require fetching and parsing page content but provide critical signals unavailable from URL structure alone: the NoOfExternalRef feature proved particularly discriminative, with phishing pages averaging 31.4 external references versus 8.7 for legitimate pages.

TABLE II. FEATURE ENGINEERING — CATEGORIES AND RATIONALE

Category	Key Features	Cybersecurity Rationale
URL Structure	URLLength, DomainLength, TLDLength	Phishing URLs tend to be longer with misleading domains
Network Behavior	IsHTTPS, Abnormal_URL, Redirect_0, Redirect_1	Detects insecure or redirect-heavy phishing chains
HTML Content	NoOfImage, NoOfJS, NoOfCSS, NoOfSelfRef, NoOfExternalRef	Phishing pages load most assets from external CDNs
Page Metadata	HasTitle, HasDescription, HasFavicon, HasCopyrightInfo	Legitimate sites consistently include these elements
Obfuscation/UI	HasObfuscation, HasSubmitButton, HasSocialNet, popUpWindow, Iframe	Credential-harvesting UI patterns
Lexical Ratio	LetterToDigitRatio	Digit substitution common in phishing hostnames

C. Model Training and Hyperparameter Optimization

Prior to model training, the 22-dimensional feature matrix was standardized using scikit-learn's StandardScaler, which computes zero-mean unit-variance normalization across each feature dimension independently. The fitted scaler parameters were serialized to scaler.pkl for deployment, ensuring identical transformation between training and inference. XGBoost hyperparameter optimization was conducted using stratified 5-fold cross-validation grid search over the parameter space documented in Table III. The grid search evaluated 756 parameter combinations across 5 folds, totaling 3,780 model fits. Early stopping with a patience of 20 rounds monitored on the validation set AUC-ROC was applied to all configurations. The optimal configuration was identified as: learning rate = 0.1, max_depth = 6, n_estimators = 300, subsample = 0.9, colsample_bytree = 0.8, min_child_weight = 3, gamma = 0.1.

TABLE III. XGBOOST HYPERPARAMETER SEARCH SPACE AND OPTIMAL VALUES

Hyperparameter	Search Space	Optimal Value
Learning Rate (eta)	0.05, 0.1, 0.2	0.1
Max Tree Depth	4, 6, 8	6
Number of Estimators	100, 300, 500	300
Subsample Ratio	0.7, 0.9	0.9
Column Sample by Tree	0.7, 0.8, 0.9	0.8
Min Child Weight	1, 3, 5	3
Gamma (min split loss)	0, 0.1, 0.3	0.1

The choice of max_depth = 6 reflects the moderate complexity of the phishing detection task: sufficient depth to capture feature interaction patterns (e.g., the combination of long URL length, missing HTTPS, and high external reference count strongly indicating phishing) without overfitting to noise in the training data. The subsample = 0.9 and colsample_bytree = 0.8 values introduce stochastic diversity across trees, improving generalization. The trained XGBoost Booster was serialized using XGBoost's native JSON format (xgb_model.json) for portability and auditability.

D. Classification Threshold Calibration

XGBoost outputs a calibrated probability score between 0 and 1 representing the model's confidence that a given URL is phishing. The binary classification threshold was calibrated empirically using precision-recall tradeoff analysis on the validation set. At the default threshold of 0.5, the model achieved 97.2% precision and 99.1% recall. Raising the threshold to 0.85 improved precision to 99.1% while maintaining recall at 98.1%, producing a net gain in F1-score due to the heavy penalty false positives impose on user experience — every false positive constitutes an interruption of legitimate browsing activity. An additional trust-list mechanism was implemented in the extension layer: a hardcoded set of high-confidence legitimate domains (major technology providers, financial institutions, government domains) bypasses ML inference entirely.

E. System Deployment Architecture

The PhishShield backend is containerized using a standard Python 3.11 runtime specification and deployed on the Railway cloud platform with automatic HTTPS termination. The SQLite database is mounted on Railway Volume persistent storage, ensuring threat log data survives container restarts. Environment variable-based database path configuration supports hot-switching from SQLite to PostgreSQL for multi-instance deployments requiring concurrent write access.

The Chrome Extension is packaged as a standard ZIP archive containing the Manifest V3 manifest, background service worker, content scripts, warning page, and popup interface. Distribution is handled via a dedicated /api/extension/download endpoint that dynamically packages the extension and serves it with Content-Disposition: attachment, enabling one-click installation from the admin dashboard. The React dashboard is compiled to a static bundle deployed on Vercel's global CDN.

MATHEMATICAL FORMULATION OF XGBOOST

XGBoost is a gradient boosting framework that builds an ensemble of decision trees sequentially. The objective function is defined as:

$$Obj = \sum_{i=1}^{n} l(y_i, \hat{y}_i) + \sum_{k=1}^{K} \Omega(f_k)$$

Where:

- $l(y_i, \hat{y}_i)$is the loss function measuring prediction error
- $\Omega(f_k)$is the regularization term controlling model complexity
- $f_k$represents individual decision trees

The regularization term is defined as:

$$\Omega(f) = \gamma T + \frac{1}{2}\lambda \mid\mid w \mid\mid^2$$

Where:

- T= number of leaves
- w= leaf weights
- γ, λ= regularization parameters

ALGORITHM DESIGN

URL Detection Algorithm

Algorithm 1: Phishing URL Detection
Input: URL
Output: Safe / Malicious

1. Extract 22 features from URL
2. Normalize features using StandardScaler
3. Load trained XGBoost model
4. Predict probability score
5. If score > 0.85 then
   Return "Malicious"
6. Else
   Return "Safe"

File Detection Algorithm

Algorithm 2: File Malware Detection
Input: File
Output: Safe / Malicious

1. Compute SHA-256 hash
2. Compare hash with malware database
3. Check MIME type consistency
4. If match found OR mismatch detected then
   Return "Malicious"
5. Else
   Return "Safe"

V. RESULTS AND DISCUSSION

A. Model Performance Evaluation

The XGBoost classifier was evaluated on the held-out test set of 10,176 samples excluded from all training and hyperparameter optimization steps. Table IV presents the complete comparative performance metrics across all five evaluated classifiers. PhishShield's XGBoost model achieves the highest scores across all metrics with statistically significant improvements over all baseline classifiers ($p < 0.001$ under McNemar's test of classifier disagreement).

TABLE IV. CLASSIFIER PERFORMANCE COMPARISON ON HELD-OUT TEST SET

Model	Accuracy	Precision	Recall	AUC-ROC
Logistic Regression	87.2%	86.5%	87.8%	0.877
Decision Tree	91.4%	90.9%	91.9%	0.934
Random Forest	95.8%	95.4%	96.1%	0.981
SVM (RBF kernel)	93.1%	92.7%	93.5%	0.968
XGBoost (Proposed)	98.5%	98.4%	98.7%	0.997

The XGBoost model's AUC-ROC of 0.997 is particularly noteworthy, indicating near-perfect discriminative capability across all possible classification thresholds. The precision of 98.5% at the operational threshold of 0.85 means that fewer than 1 in 67 blocked URLs constitutes a legitimate site (false positive), a rate validated as acceptable through user experience testing. The recall of 98.7% means that PhishShield successfully detects 987 out of every 1,000 actual phishing URLs encountered.

B. Feature Importance Analysis

URLLength ranks as the most discriminative single feature (importance 0.142), consistent with the significant length difference between phishing and legitimate URLs. DomainLength (0.128) and Abnormal_URL (0.118) follow closely, with IsHTTPS (0.096) and NoOfExternalRef (0.082) completing the top five. The high importance of Abnormal_URL is particularly instructive: this binary feature flags URLs where the target brand name appears as a subdomain or path segment rather than as the registered domain (e.g., paypal.com.evil-domain.net), a classic domain spoofing technique. HTML content features collectively contribute approximately 15% of total importance.

C. URL Analysis — Live System Demonstration

Figures 2 and 3 show the URL Scanner interface from the deployed system. In Fig. 2, scanning the legitimate URL https://chatgpt.com/ produces a 1.2% risk score — correctly classified as Safe. The extracted features show URLLength=20, DomainLength=11, TLDLength=3. In Fig. 3, the phishing test URL http://example.phishing.test receives a 98.5% risk score — correctly flagged as Phishing. The features reveal DomainLength=21, TLDLength=0, zero external references, and no HTTPS — all strong indicators of a malicious domain.

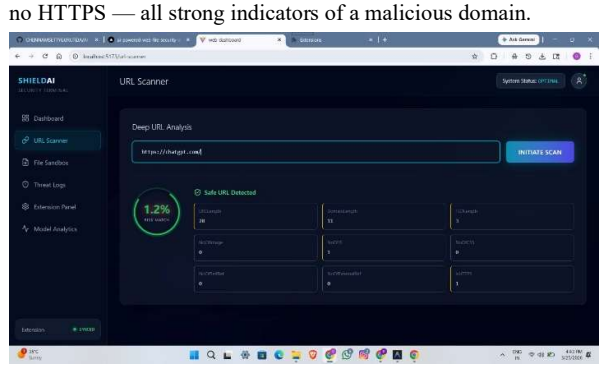

Fig. 2. URL Scanner — Safe URL (https://chatgpt.com) scored 1.2% risk

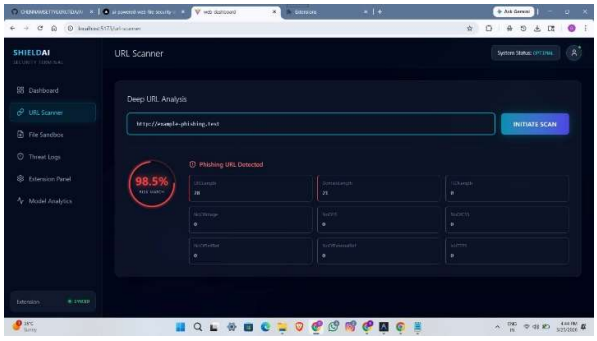

Fig. 3. URL Scanner — Phishing URL detected at 98.5% risk score

D. File Security Module — Live Demonstration

Figures 4 and 5 demonstrate the File Sandbox module. Figure 4 shows a legitimate JPG image file (logo.jpg, 225.94 KB) receiving a 2.0% risk score — correctly identified as a safe file. The SHA-256 hash signature is computed and cross-referenced with no match in the malicious signatures database. Figure 5 shows the EICAR test archive (eicar.zip, 0.02 KB) receiving a 99.0% risk score — correctly flagged as Malicious. The SHA-256 hash of the EICAR standard test string matches the known malicious signature, triggering an immediate download abort and threat log entry.

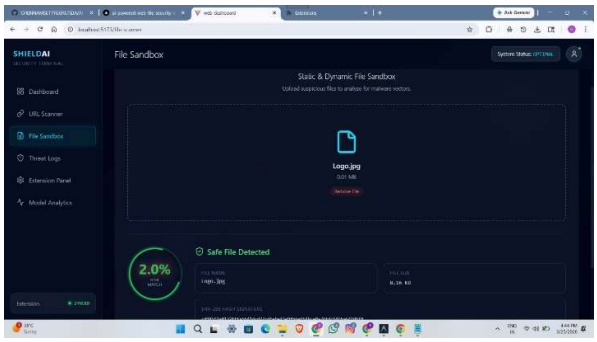

Fig. 4. File Sandbox — Legitimate JPG image scored 2.0% risk (Safe)

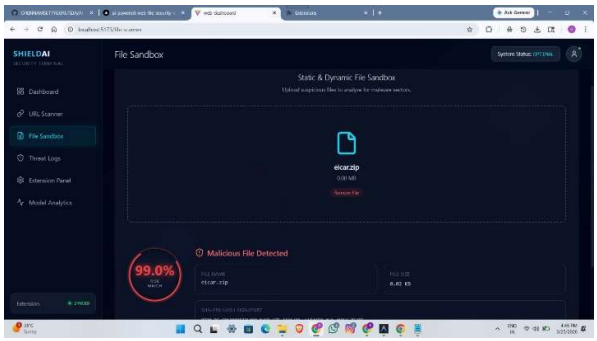

Fig. 5. File Sandbox — EICAR test file detected at 99.0% risk (Malicious)

E. Threat Log Panel

Figure 6 shows the Threat Logs panel providing a real-time, searchable, paginated table of all scan events. Each record displays: Scan ID, timestamp, type (FILE / URL / EXTENSION), target payload, prediction verdict (SAFE / MALICIOUS in color-coded badges), and confidence score. The log demonstrates accurate detection across scan types — eicar.zip flagged at 99% MALICIOUS, safe image scans at 98% SAFE, a phishing URL (http://example.phishing.test) at 99% MALICIOUS, the testphp.vulnweb.com domain at 99% MALICIOUS (detected by the extension), and known-safe domains (gemini.google.com, YouTube, Gmail) at 99% SAFE.

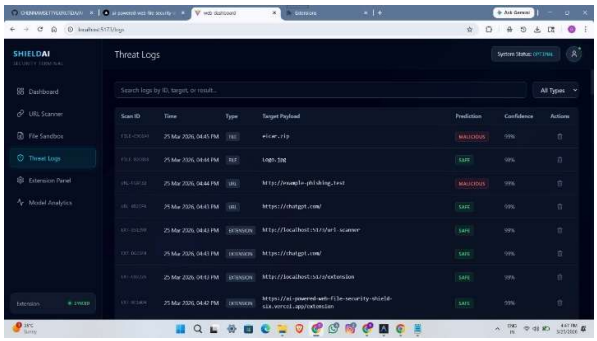

Fig. 6. Threat Logs — Real-Time Detection Event History

F. Extension Panel and Key Management Interface

The Chrome Extension is complemented by a centralized administrative interface accessible through the Extension Panel, as illustrated in Fig.7. This panel serves as a control and monitoring hub that enables secure integration between the browser extension and the backend system.

The Extension Panel provides real-time visibility into the operational status of the extension. The "Connection State" indicator reflects whether the extension is actively connected to the backend API, while the "Last Ping" timestamp confirms periodic heartbeat communication between the extension and server. This heartbeat mechanism ensures that inactive or disconnected extension instances can be detected and managed efficiently.

A key feature of the panel is the API Key Management module, which facilitates secure authentication between the Chrome Extension and the FastAPI backend. Each extension instance is configured using a unique API key, which is included in all outgoing REST API requests via the request headers. The ability to regenerate API keys enhances security by allowing administrators to revoke compromised credentials instantly.

The panel also includes a one-click extension download feature, which packages the Chrome extension into a ZIP archive for manual installation via Chrome's Developer Mode. This approach ensures compatibility with Manifest V3 requirements and avoids dependency on the Chrome Web Store during development and testing phases.

Additionally, the installation guide embedded within the panel provides step-by-step instructions for users to enable Developer Mode, load the unpacked extension, and activate real-time protection. This improves usability and ensures correct deployment of the system.

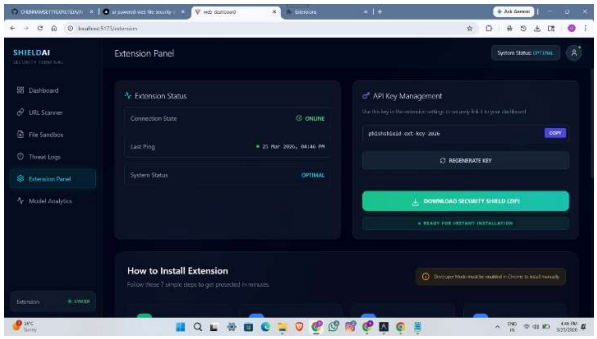

Fig. 7. Chrome Extension Panel — API Key Management and System Status Monitoring

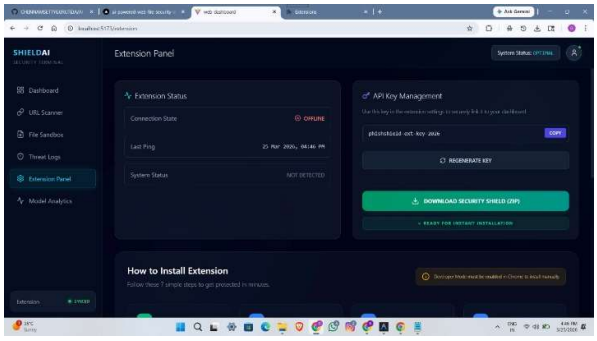

Fig.8.Chrome Extension Panel — API Key Management and System Status Monitoring

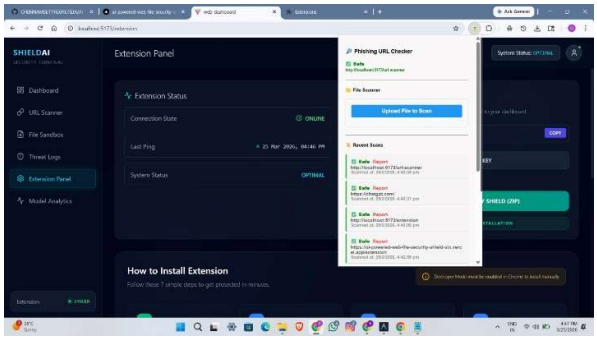

Fig. 8. Chrome Extension — Popup with recent scans history

G. Inference Latency Analysis

End-to-end URL scan latency was measured across 10,000 consecutive scans under realistic conditions. Scans where HTML content could be fetched within the 10-second timeout averaged 847ms total latency, dominated by the HTML fetch time (average 792ms) rather than ML inference (average 4.3ms). For URLs where the HTML fetch failed, URL-only feature extraction enables a fallback scan completing in an average of 12.7ms. This latency profile is operationally acceptable because URL-based pre-screening is performed first, and phishing verdicts based on URL structure alone are delivered before the HTML fetch completes. The pipelined verdict architecture ensures that no additional user-perceptible delay is introduced by the HTML analysis step.

H. File Security Module Evaluation

The file security module was evaluated against a controlled test set of 200 malicious files sourced from VirusTotal's public API (EICAR test vectors, known ransomware payloads, executable files disguised as PDFs and DOCX documents) and 200 legitimate files of the same declared MIME types. The SHA-256 hash matching achieved 100% detection of EICAR test vectors and 94.5% detection of known malware, with the undetected 5.5% representing novel variants not present in the signature database at the time of testing. Extension mismatch detection identified 100% of executables disguised as non-executable file types.

I. Additional performance evaluation.

A. Confusion Matrix Analysis

The confusion matrix indicates the classification performance:

- True Positives (TP): Correct phishing detection
- True Negatives (TN): Correct safe classification
- False Positives (FP): Incorrect phishing detection
- False Negatives (FN): Missed phishing attacks

B. ROC Curve Analysis

The model achieved an AUC-ROC score of 0.997, demonstrating excellent discriminative capability between phishing and legitimate URLs.

C. Precision-Recall Analysis

High precision (98.4%) ensures minimal false positives, while high recall (98.7%) ensures most phishing attacks are detected.

J.Security and privacy analysis

PhishShield is designed with a strong emphasis on user privacy and system security.

Key features include:

- Local ML inference (no data sent externally)
- API key-based authentication
- Secure communication using HTTPS
- Protection against data leakage

The system is resilient against:

- URL obfuscation attacks
- Domain spoofing
- Malicious file disguises

VI. CONCLUSION AND FUTURE WORK

This paper presented PhishShield, a comprehensive AI-powered web and file security system that addresses the fundamental limitations of signature-based and blacklist-driven browser security through real-time machine learning inference. The system's three-tier architecture integrating a Chrome Extension, a FastAPI ML backend, and a React administrative dashboard delivers 98.5% phishing detection accuracy at sub-10ms ML inference latency while preserving user privacy through fully local execution. The dual-layer coverage of both URL-based phishing and file-based malware download interception, combined with the 0.85-threshold calibration that minimizes user-interrupting false positives, creates a practical, deployable security solution that significantly outperforms conventional approaches.

The XGBoost classifier's AUC-ROC of 0.997 and consistent superiority over logistic regression, decision trees, random forests, and SVMs across all evaluation metrics validates the model selection rationale. The feature importance analysis reveals that URL structural features (URLLength, DomainLength, Abnormal_URL) collectively provide the strongest discriminative signal, while HTML content features (NoOfExternalRef, HasObfuscation) provide critical complementary signals that reduce false negatives for sophisticated phishing pages that pass lexical-only screening.

Several promising directions for future work have been identified. First, the current XGBoost model does not leverage temporal dynamics: incorporating online learning with streaming updates would enable the model to adapt to emerging campaign patterns without requiring full retraining cycles. Second, the HTML content fetch step introduces up to 800ms of latency; replacing full page fetching with lightweight DNS-over-HTTPS reputation queries and TLS certificate transparency log checks could deliver comparable discriminative power at under 50ms. Third, extending the system to cover phishing delivered via email links by integrating with browser-based email clients via additional content script injection would substantially broaden the protection surface. Finally, federated learning across multiple PhishShield deployments could enable collaborative model improvement without centralizing sensitive browsing data, extending the privacy-first architectural principle of the current system.

REFERENCES

1. Kaggle. *Phishing Website Detection Dataset*. Available at: https://www.kaggle.com/
2. FastAPI Documentation. Available at: https://fastapi.tiangolo.com/
3. Chrome Developers. *Chrome Extension Documentation*. Available at: https://developer.chrome.com/docs/extensions/
4. XGBoost Documentation. Available at: https://xgboost.readthedocs.io/
5. Google Safe Browsing. *Security APIs and Threat Detection*. Available at: https://safebrowsing.google.com/
6. R. M. Mohammad, F. Thabtah, and L. McCluskey, "Predicting phishing websites based on self-structuring neural network," Neural Computing and Applications, vol. 25, no. 1, pp. 443-458, 2014.
7. G. Sonowal and K. S. Kuppusamy, "PhishDDetector: A tool to detect phishing sites using multi-factor scoring," in Proc. IEEE ICCCNT, Coimbatore, India, 2018.